Raspodijeljeni sustavi (2026/2027)

VJEŽBA 2: Priprema podataka za web-aplikaciju

U prethodnoj vježbi izradili ste jednostavnu web-aplikaciju koja se sastoji od **frontenda** i **REST API poslužitelja**, koje ste odvojeno deployali na cloud servis Render.com.

Web-aplikacija često treba podatke iz nekoliko različitih izvora prije nego što ih može prikazati korisniku. Primjerice, aplikacija restorana može dohvaćati jelovnik, recenzije gostiju i informacije o slobodnim stolovima. Ako su ove operacije međusobno neovisne, možemo ih izvršavati **konkurentno**.

U ovoj vježbi nadogradit ćete prethodni projekt zasebnim Python programom koji će **simulirati dohvaćanje i pripremu podataka** za vašu aplikaciju. Usporedit ćete sekvencijalno i konkurentno izvođenje, a pripremljene podatke prikazati na postojećem frontendu.

Predznanje - studenti moraju ponoviti sljedeće gradivo:

- sinkroni Python: funkcije, liste, rječnici, petlje i *comprehension* sintaksa (`RS1`, `RS2`)
- korutine, `async` i `await`, *event loop*, `asyncio.sleep()`, `asyncio.gather()` i `asyncio.create_task()` (`RS3`)
- struktura vlastite web-aplikacije izrađene u prethodnoj vježbi

> Važno! Dohvaćanje podataka u ovoj vježbi simuliramo pomoću `asyncio.sleep()`. Python program zasad ne šalje stvarne HTTP zahtjeve. Podatke definirajte unutar programa, a stvarnu mrežnu komunikaciju dodat ćemo na narednim vježbama.

> Za repetitivan posao i prilagodbu frontenda možete koristiti AI agente po izboru. Međutim, morate znati objasniti redoslijed izvođenja korutina, način prikupljanja rezultata i dobivena vremena izvođenja.

Zadatak 1

→ 0.5 bodova

Izradite novi direktorij unutar postojećeg repozitorija za RS vježbe. Unutar njega izradite Python program koji će **pripremati sintetičke podatke** povezane s **temom vaše aplikacije iz prethodne vježbe**.

Definirajte **barem tri korutine** koje **simuliraju** dohvaćanje različitih podataka iz vanjskih izvora, primjerice baza podataka. Svaka korutina mora:

- ispisati poruku o početku dohvaćanja
- simulirati čekanje pomoću `asyncio.sleep()`
- ispisati poruku o završetku dohvaćanja
- vratiti podatke u obliku liste ili rječnika

Svaka korutina neka sadrži nasumično čekanje između **1 do 3 sekunde** (*hint*: modul `random`). Podatke osmislite ili generirajte sami; svaka korutina mora vratiti konkretne podatke koje ćete kasnije prikazati na frontendu.

Primjer za aplikaciju restorana:

Korutina	Vrijeme čekanja	Podaci koje vraća
`dohvati_jelovnik()`	Nasumično, 1–3 sekunde	Lista jela s identifikatorom, nazivom i cijenom
`dohvati_recenzije()`	Nasumično, 1–3 sekunde	Lista recenzija s imenom gosta, ocjenom i komentarom
`dohvati_stolove()`	Nasumično, 1–3 sekunde	Lista stolova s oznakom i brojem slobodnih mjesta

Primjeri: Ako ste izradili webshop, možete dohvaćati proizvode, recenzije i opcije dostave. Za turističku aplikaciju možete dohvaćati smještaj, događanja i vremensku prognozu.

Važno! Odabrane operacije moraju biti međusobno neovisne: nijednoj nisu potrebni rezultati druge operacije da bi se mogla izvršiti.

Zadatak 2

→ 1 bod

Korutine iz prethodnog zadatka izvršite na **dva različita načina**:

1. **Sekvencijalno**
2. **Konkurentno**

Za obje verzije izmjerite ukupno vrijeme izvođenja. Mjerenje mora obuhvatiti dohvaćanje svih skupina podataka.

Za usporedbu koristite **ista vremena čekanja za odgovarajuće korutine u obje verzije**. Nasumična vremena možete generirati jednom prije mjerenja i proslijediti ih korutinama.

Rezultate svake verzije **objedinite u rječnik iste strukture**, primjerice s ključevima `jelovnik`, `recenzije` i `stolovi`.

Primjer:

```
{
    "jelovnik": [
        {"id": 1, "naziv": "Pizza Margherita", "cijena": 9.50},
        {"id": 2, "naziv": "Rižoto s gljivama", "cijena": 12.00}
    ],
    "recenzije": [
        {"gost": "Ana", "ocjena": 5, "komentar": "Odlična pizza!"},
        {"gost": "Marko", "ocjena": 4, "komentar": "Ugodna atmosfera."}
    ],
    "stolovi": [
        {"oznaka": "T1", "slobodna_mjesta": 4},
        {"oznaka": "T2", "slobodna_mjesta": 2}
    ]
}
```

Odgovorite na pitanja:

- Koliko traje izvođenje programa pri sekvencijalnoj obradi? Zašto?
- Koliko traje izvođenje programa pri konkurentnoj jednodretvenoj obradi? Zašto?
- Koja je glavna razlika između ovih dvaju načina izvođenja? Kako se korutine izvršavaju unutar *event loopa*?
- Što se događa ako u korutinama pokušate zamijeniti `asyncio.sleep` sa `time.sleep`. Koja je tehnička razlika u implementaciji ovih `sleep` funkcija?

Na početku programa, prije generiranja nasumičnih vrijednosti, postavite `random.seed(42)`. Testirajte kako funkcionira `asyncio.gather(korutina_1(), korutina_2(), korutina_3(), ...)` i to sa **različitim redoslijedom navođenja parametara**.

Odgovorite na pitanje:

Vraća li `asyncio.gather()` rezultate uvijek istim redoslijedom? Odgovara li taj redoslijed redoslijedu proslijeđenih korutina ili redoslijedu njihova dovršavanja?

Zadatak 3

→ 1 bod

Objedinjeni rezultat konkurentne verzije iz prethodnog zadatka spremite u datoteku `podaci.json`, koristeći Pythonov modul `json`.

Nadogradite frontend iz prve vježbe tako da učita ovu datoteku i prikaže **sve skupine dohvaćenih podataka**.

JSON datoteku možete ručno kopirati u direktorij iz kojeg frontend poslužuje **statične datoteke** (eng. `public`, `assets`, i sl.). Prikaz prilagodite vlastitoj aplikaciji, primjerice kroz kartice, liste, tablicu...

Ponovno deployajte frontend na Render.com i provjerite prikazuju li se podaci na javnom URL-u.

Predaja vježbe

Verzionirajte Python program, generiranu datoteku `podaci.json` i `README.md` unutar novog direktorija u postojećem Git repozitoriju.

Frontend nadogradite na postojećoj lokaciji iz prve vježbe; **nije potrebno izrađivati novu frontend aplikaciju**.

U `README.md` datoteku dodajte:

- kratak opis podataka i operacija koje simulirate
- korištena vremena čekanja i izmjerena vremena sekvencijalnog i konkurentnog izvođenja
- odgovore na pitanja iz drugog zadatka
- javni URL nadograđene frontend aplikacije

Kada završite s vježbom, učitajte na Merlin link na vaš Git repozitorij. Ako vam treba pomoć, ili želite komentar na zadaću, javite se na Chat ili ostavite komentar na Merlinu prilikom predavanja zadaće.